“Notificador Express”: usando uma interface para enviar mensagens

Contexto
Uma equipe precisa enviar notificações por diferentes canais (e-mail, SMS, etc.). Eles querem poder trocar o canal sem mudar o código que chama o envio.

Objetivo
- Criar uma **interface** com um método comum.
- Implementar **duas classes** concretas.
- Mostrar **polimorfismo via interface** (troca de implementação em tempo de execução).

O que implementar
1. **Interface Notificador**

```
public interface Notificador {
    void enviar(String destinatario, String mensagem);
}
```

2. **Implementações**
- EmailNotificador (simular envio imprimindo no console: “Enviando EMAIL para ...: ...”).
- SmsNotificador (simular envio: “Enviando SMS para ...: ...”).

3. **Classe de demonstração App**
- Ler (ou fixar) um “canal” (ex.: "email" ou "sms").
- Com base no canal, instanciar a implementação correspondente **através da referência do tipo Notificador**.
- Chamar enviar(...).

Regras / restrições
- Nada de bibliotecas externas; só System.out.println.
- O App deve **depender da interface**, não das classes concretas.

Passo a passo sugerido (rápido)
1. Criar Notificador.
2. Criar EmailNotificador e SmsNotificador.
3. Em App, ter algo como:
4. Notificador n;
5. String canal = "email"; // ou "sms"
6. if (canal.equals("email")) n = new EmailNotificador();
7. else n = new SmsNotificador();
8. n.enviar("aluno@exemplo.com", "Bem-vindo ao curso!");
9. Executar e mostrar a troca de implementação mudando o valor de canal.

Exemplo esperado (console)

```
Enviando EMAIL para aluno@exemplo.com: Bem-vindo ao curso!
```

Ou

```
Enviando SMS para +55 11 99999-9999: Bem-vindo ao curso!
```